PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

DOCUMENTO TÉCNICO: DESARROLLO DEL FRONTEND CON REACT Y CONSUMO DE API CON EXPRESS

Presentado a: **Instructora Yoly Alexandra Muñoz**

Por Aprendiz: **Cristian David Montilla Ordoñez**

Ficha: 3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día 21 de octubre del año 2026

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

1. INTRODUCCIÓN

En esta actividad desarrollé un frontend con React para comunicarme con una API creada con Express. El objetivo fue consultar información del servidor, mostrar los datos recibidos y enviar información mediante un formulario.

Creé tres componentes llamados Inicio, Informacion y Solicitudes. Cada uno permite consultar un mensaje, mostrar información general o enviar un nombre, apellido y correo electrónico al backend.

Durante el desarrollo utilicé useState, fetch(), async, await y response.json() para realizar las peticiones y mostrar las respuestas. Con este ejercicio aprendí cómo se comunican React y Express y la importancia de utilizar correctamente los nombres de los datos.

2. OBJETIVO

2.1. Objetivo general

Desarrollar un frontend en React que permita consultar información y enviar solicitudes a una API creada con Express, utilizando peticiones HTTP y mostrando las respuestas recibidas en la interfaz.

2.2. Objetivos específicos

Crear componentes independientes para consultar los endpoints /inicio y /informacion, y enviar datos a /solicitudes.

Aprender a utilizar fetch(), async y await para comunicarse con el backend.

Utilizar useState para guardar los datos y mostrar los resultados en pantalla.

Crear un formulario que permita enviar nombre, apellido y correo electrónico.

Realizar pruebas para comprobar que los componentes se comunican correctamente con la API.

3. INSTALACIÓN Y CONFIGURACIÓN

3.1. ¿Para qué sirven Node.js y npm?

Node.js es una herramienta que permite ejecutar JavaScript fuera del navegador. La utilicé para trabajar con las herramientas necesarias para desarrollar el proyecto.

npm permite instalar y administrar los paquetes que necesita la aplicación. Por ejemplo, mediante npm se instalan las dependencias de React y Vite para poder ejecutar el frontend.

Estas herramientas son importantes porque permiten preparar el entorno de trabajo y ejecutar los comandos necesarios para desarrollar la aplicación.

3.2. Creación del proyecto con Vite

Para crear el proyecto utilicé Vite, una herramienta que facilita la creación y ejecución de aplicaciones frontend.

El proceso consiste en abrir una terminal y ejecutar el comando:

```
npm create vite@latest
```

Después se siguen las instrucciones para asignar el nombre del proyecto y seleccionar React con JavaScript.

Luego se ingresa a la carpeta del proyecto y se instalan las dependencias:

```
cd nombre-del-proyecto
```

```
npm install
```

Finalmente, se inicia el servidor de desarrollo con:

```
npm run dev
```

La terminal muestra una dirección local que se puede abrir en el navegador para visualizar la aplicación. Cada vez que se realizan cambios y se guarda el archivo, Vite permite ver las modificaciones durante el desarrollo.

3.3. Archivos importantes del proyecto

Los principales archivos y carpetas que utilicé son los siguientes:

package.json: contiene información del proyecto, las dependencias instaladas y los comandos disponibles para ejecutar la aplicación.

node_modules: es la carpeta donde se encuentran los paquetes instalados mediante npm.

index.html: es el archivo HTML principal que utiliza Vite para cargar la aplicación.

src/main.jsx: es el archivo que inicia la aplicación React y permite mostrar el componente principal.

src/App.jsx: es el componente principal donde puedo importar y mostrar los componentes que desarrollé.

src/Inicio.jsx: contiene el componente encargado de consultar el endpoint /inicio.

src/Informacion.jsx: contiene el componente que consulta /informacion y muestra las propiedades recibidas.

src/Solicitudes.jsx: contiene el formulario que permite enviar los datos al endpoint /solicitudes.

Estos archivos cumplen funciones diferentes, pero trabajan juntos para mostrar la aplicación en el navegador.

3.4. Ejecución y comprobación del backend

Para comprobar el funcionamiento del proyecto también es necesario iniciar el servidor Express. En mi caso, el backend trabaja en el puerto 2000.

Utilicé el comando configurado para iniciar el servidor:

npm start

Después comprobé la ruta de inicio utilizando la dirección:

http://127.0.0.1:2000/inicio

Si Express está funcionando y la ruta está definida correctamente, debe devolver el mensaje configurado en el backend.

También comprobé la ruta /informacion, que devuelve las propiedades servicios, version y descripcion.

El frontend se ejecuta con Vite en su puerto de desarrollo, mientras que Express utiliza el puerto 2000. Es importante que ambos servidores estén funcionando para realizar las pruebas de comunicación.

4. DESARROLLO DE LOS COMPONENTES

4.1. Componente Inicio

El componente Inicio tiene la responsabilidad de consultar el endpoint GET /inicio y mostrar en pantalla el mensaje que devuelve Express.

La petición se ejecuta cuando el usuario presiona el botón. De esta manera, la consulta no se realiza automáticamente al abrir la aplicación.

Endpoint utilizado: http://127.0.0.1:2000/inicio

Método HTTP: GET.

Funcionamiento:

Primero, el usuario presiona el botón para llamar a la API. Después se ejecuta la función que contiene fetch() y se espera la respuesta con await. Cuando llega la respuesta, se utiliza response.json() para leer los datos recibidos. Finalmente, el mensaje se guarda en el estado mediante setMensaje() y React lo muestra en un párrafo.

Un fragmento del código es:

```
const [mensaje, setMensaje] = useState("")


async function consumoApi() {
    let respuesta = await fetch(
        "http://127.0.0.1:2000/inicio"
    )


    let datos = await respuesta.json()


    console.log(datos)
    setMensaje(datos.mensaje)
}
```

La variable mensaje guarda el texto que se muestra en pantalla. La función setMensaje permite actualizar ese texto cuando llega la respuesta del servidor.

Los posibles problemas incluyen que Express esté apagado, que la URL sea incorrecta o que la ruta no esté definida. Para completar el manejo de errores, se puede utilizar try y catch para mostrar un aviso si falla la comunicación, y comprobar respuesta.ok para identificar una respuesta HTTP con error. Estas comprobaciones deben agregarse al código si todavía no se encuentran implementadas.

4.2. Componente Informacion

El componente Informacion permite consultar el endpoint GET /informacion. Su responsabilidad es mostrar los datos generales que devuelve la API.

Endpoint utilizado: http://127.0.0.1:2000/informacion

Método HTTP: GET.

La información que devuelve Express contiene tres propiedades: servicios, version y descripcion.

En el componente utilicé tres estados:

```
const [servicios, setServicios] = useState("")

const [version, setVersion] = useState("")

const [descripcion, setDescripcion] = useState("")
```

Cada estado guarda una parte de la información recibida.

Después de realizar la petición y convertir la respuesta a JSON, utilizo las siguientes instrucciones:

setServicios(datos.servicios)

setVersion(datos.version)

setDescripcion(datos.descripcion)

Estas instrucciones guardan las propiedades recibidas en sus respectivos estados.

En el apartado visual, cada dato se muestra mediante un párrafo:

{servicios}

{version}

{descripcion}

Cuando React recibe los nuevos valores de los estados, actualiza la información que aparece en pantalla.

Los problemas que pueden presentarse son una ruta incorrecta, un servidor apagado o una respuesta que no tenga las propiedades esperadas. Para manejar estos casos se puede agregar una comprobación de la respuesta y utilizar try y catch para mostrar mensajes comprensibles.

4.3. Componente Solicitudes

El componente Solicitudes permite ingresar nombre, apellido y correo electrónico, y enviar esos datos al endpoint POST /solicitudes.

Endpoint utilizado: http://127.0.0.1:2000/solicitudes

Método HTTP: POST.

Para capturar los datos utilizo useState y el evento onChange. Cuando el usuario escribe en un campo, el estado correspondiente se actualiza con el valor ingresado.

Por ejemplo:

const [nombre, setNombre] = useState("")

const [apellido, setApellido] = useState("")

const [correo, setcorreo] = useState("")

Cada variable guarda el contenido de uno de los campos del formulario.

Para enviar los datos utilizo fetch() con el método POST:

let resultado = await fetch(

```
  "http://127.0.0.1:2000/solicitudes",
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      nombre: nombre,
      apellido: apellido,
      correo: correo
    })
  }
)
```

En este código, method indica el tipo de petición. La cabecera Content-Type indica que el contenido se envía en formato JSON. JSON.stringify() convierte el objeto de JavaScript en un texto que se puede enviar al servidor.

Express recibe los datos y ejecuta la lógica definida en la ruta /solicitudes. Después devuelve una respuesta que React puede leer y mostrar.

Es importante tener en cuenta que enviar los datos no significa que se guarden permanentemente. En este proyecto, el almacenamiento depende de lo que se haya programado en el backend. Si no se utiliza una base de datos ni otro método de almacenamiento, recibir la solicitud no garantiza que los datos queden guardados.

Manejo de errores y validaciones

La actividad pide comprobar el estado de la respuesta, mostrar mensajes de éxito o rechazo y avisar cuando no se pueda establecer comunicación con el servidor.

Para cumplir ese requisito se debe comprobar respuesta.ok después de recibir la respuesta HTTP. También se puede utilizar try y catch para manejar errores de conexión o problemas al leer la respuesta.

Por ejemplo, la lógica básica es:

```
let datos = await resultado.json()


if (resultado.ok) {
```

```
    setMensaje(datos.mensaje)
} else {
    setMensaje("La API rechazó los datos")
}
```

En el componente completo, este código debe estar dentro de un try y acompañado de un catch que muestre un mensaje cuando no se pueda realizar la comunicación.

La propiedad datos.mensaje debe coincidir con la respuesta real de Express. Si el backend devuelve otras propiedades, como dato_nombre, dato_apellido y dato_correo, el frontend debe utilizar esos nombres cuando corresponda.

El botón se utiliza para ejecutar la función de envío. Como la versión desarrollada utiliza onClick y no un formulario enviado mediante onSubmit, el botón normal no provoca por sí solo una recarga de la página.

5. EXPLICACIÓN DE LAS FUNCIONES UTILIZADAS

5.1. useState

useState permite guardar un valor dentro de un componente React y actualizarlo cuando sea necesario.

En mi proyecto lo utilicé para guardar el mensaje recibido de Express, los datos generales de la API y los valores escritos por el usuario en el formulario.

Por ejemplo:

```
const [mensaje, setMensaje] = useState("")
```

mensaje es la variable que contiene el valor actual y setMensaje es la función que permite cambiarlo. Cuando el estado cambia, React actualiza la parte de la interfaz que utiliza ese valor.

5.2. fetch()

fetch() permite realizar una petición a una dirección. En este proyecto se utiliza para consultar información mediante GET y enviar datos mediante POST.

Por ejemplo:

```
fetch("http://127.0.0.1:2000/inicio")
```

Esta instrucción realiza la petición al endpoint /inicio. Para enviar datos también se pueden indicar opciones como el método HTTP, las cabeceras y el cuerpo de la petición.

5.3. async y await

async se utiliza para declarar una función asíncrona y await permite esperar el resultado de una operación antes de continuar con las siguientes instrucciones.

Los utilicé porque las peticiones a la API pueden tardar un tiempo en responder. Así puedo esperar la respuesta y después procesar los datos recibidos.

5.4. response.json()

response.json() permite leer el contenido de una respuesta JSON y convertirlo en un objeto de JavaScript.

En mis componentes lo utilicé para acceder a propiedades como mensaje, servicios, version y descripcion.

Por ejemplo:

```
let datos = await respuesta.json()
```

Después de esa conversión puedo utilizar instrucciones como datos.mensaje para obtener un valor específico.

5.5. JSON.stringify()

JSON.stringify() convierte un objeto de JavaScript en un texto con formato JSON.

Lo utilicé en el componente Solicitudes para preparar los datos antes de enviarlos mediante POST.

```
JSON.stringify({
    nombre: nombre,
    apellido: apellido,
    correo: correo
})
```

Así, el backend puede interpretar el cuerpo de la petición cuando tiene configurado el middleware correspondiente para recibir JSON.

5.6. Eventos y manejo de errores

onClick permite ejecutar una función cuando el usuario presiona un botón. Lo utilicé para iniciar las consultas y el envío de los datos.

onChange permite detectar cambios en los campos de entrada. En Solicitudes se utiliza para guardar lo que escribe el usuario.

onSubmit se utiliza cuando se desea ejecutar una función al enviar un formulario. Cuando se utiliza ese evento, preventDefault() permite evitar que el navegador recargue la página por el envío normal del formulario.

Para manejar errores, try permite intentar ejecutar una operación y catch permite responder cuando ocurre un error durante su ejecución. Además, respuesta.ok sirve para revisar si el servidor devolvió un código HTTP entre 200 y 299. Esto es importante porque fetch() puede recibir una respuesta con código de error sin que ocurra un error de conexión.

6. PRUEBAS Y RESULTADOS

Las evidencias ya están en el otro PDF principal

7. DIFICULTADES Y SOLUCIONES

7.1. Problema con la ruta /informacion

Al principio tuve un error 404 al consultar la ruta. Revisé que Express estuviera funcionando y que la dirección estuviera bien escrita. Después comprobé que la ruta existiera en el backend.

7.2. Diferencia en los nombres de los datos

También aprendí que los nombres de las propiedades que devuelve Express deben coincidir con los que utilizo en React. Para revisar esto, utilicé console.log(datos) y comprobé qué información estaba recibiendo.

8. CONCLUSIONES

Con esta actividad aprendí a crear componentes en React que se comunican con una API desarrollada con Express. Comprendí que React se encarga de mostrar la información con la que interactúa el usuario, mientras que Express recibe las peticiones y devuelve respuestas.

También aprendí a utilizar fetch() para consultar y enviar datos, async y await para esperar las respuestas, y response.json() para leer la información recibida. Además, utilicé useState para guardar los valores y actualizar la interfaz.

Con el formulario de Solicitudes entendí que es necesario enviar los datos con los nombres correctos, indicar el método POST y configurar la cabecera Content-Type cuando se envía JSON.

Finalmente, comprendí que recibir una respuesta del servidor no significa necesariamente que la solicitud haya sido exitosa. Por eso es importante revisar los resultados, comprobar los códigos HTTP y mostrar mensajes que ayuden al usuario a entender qué ocurrió.

Esta actividad me permitió practicar la comunicación entre frontend y backend y reconocer la importancia de probar cada componente antes de dar por terminado el proyecto.

9. REFERENCIAS

Vite. Guía de inicio para crear proyectos. https://vite.dev/guide/

React. Estado de los componentes y useState. https://react.dev/learn/state-a-components-memory

React. Responder a eventos en los componentes. https://react.dev/learn/responding-to-events

MDN Web Docs. Uso de Fetch API para realizar peticiones. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

MDN Web Docs. Funciones asíncronas en JavaScript. https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Statements/async_function

MDN Web Docs. JSON.stringify(). https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify

Express. Guía de rutas y métodos HTTP. https://expressjs.com/en/guide/routing.html

Node.js. Sitio oficial. https://nodejs.org/

npm. Documentación oficial. https://docs.npmjs.com/

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS